

PROYECTO SISTEMA MIRADOR

INGENIERIA DE SOTFWARE RQY1102

Joaquin garcia
Ignacio Reyes
Francisco adrovez

Problema/Necesidad del cliente

En el condominio el mirador hay 160 departamentos y solo 3 administrativos. Todo el control de los gastos comunes se hace a mano revisando transferencias en un excel. Esto causa un desorden tremendo en el historial, datos cruzados y que la morosidad suba porque nadie se entera a tiempo de lo que debe

Solucion

Un sibstema web transaccional y contable. Automatiza las validaciones de los pagos en tiempo real, se conecta con pasarelas de pago y calcula de forma automatica los gastos comunes e intereses de cada propiedad

Etapas del proyecto

Analisis: (levantamiento de reglas de prorrateo y deudas)

Diseño: (modelo de datos y arquitectura en capas),
Desarrollo (modulos web y Api)

Testing (pruebas de concurrencia),
Mantenimiento (monitoreo de la integracion).

Metodologia SCRUM

Justificacion:

Al trabajar con integraciones de terceros como webpay o flow y diseñar plantillas desde cero, necesitabamos un marco agil que nos permitiera hacer sprints cortos, adaptamos rapido al feedback y asegurar que los entregables funcionen rapido

Objetivo general:

Desarrollar un subsistema web transaccional para optimizar la recaudacion y bajar la morosidad en el Mirador.

Alcance:

Portal del residente (pagos y cartolas), consola del administrador (auditoria y cargos), motor de concilacion y conexion con pasarelas de pagos.

Limitaciones del proyecto

El sistema depende del uptime de las pasarelas externas (webpay/flow). Ademias, queda fuera del alcance la gestion de sueldos o contratos de los conserjes.

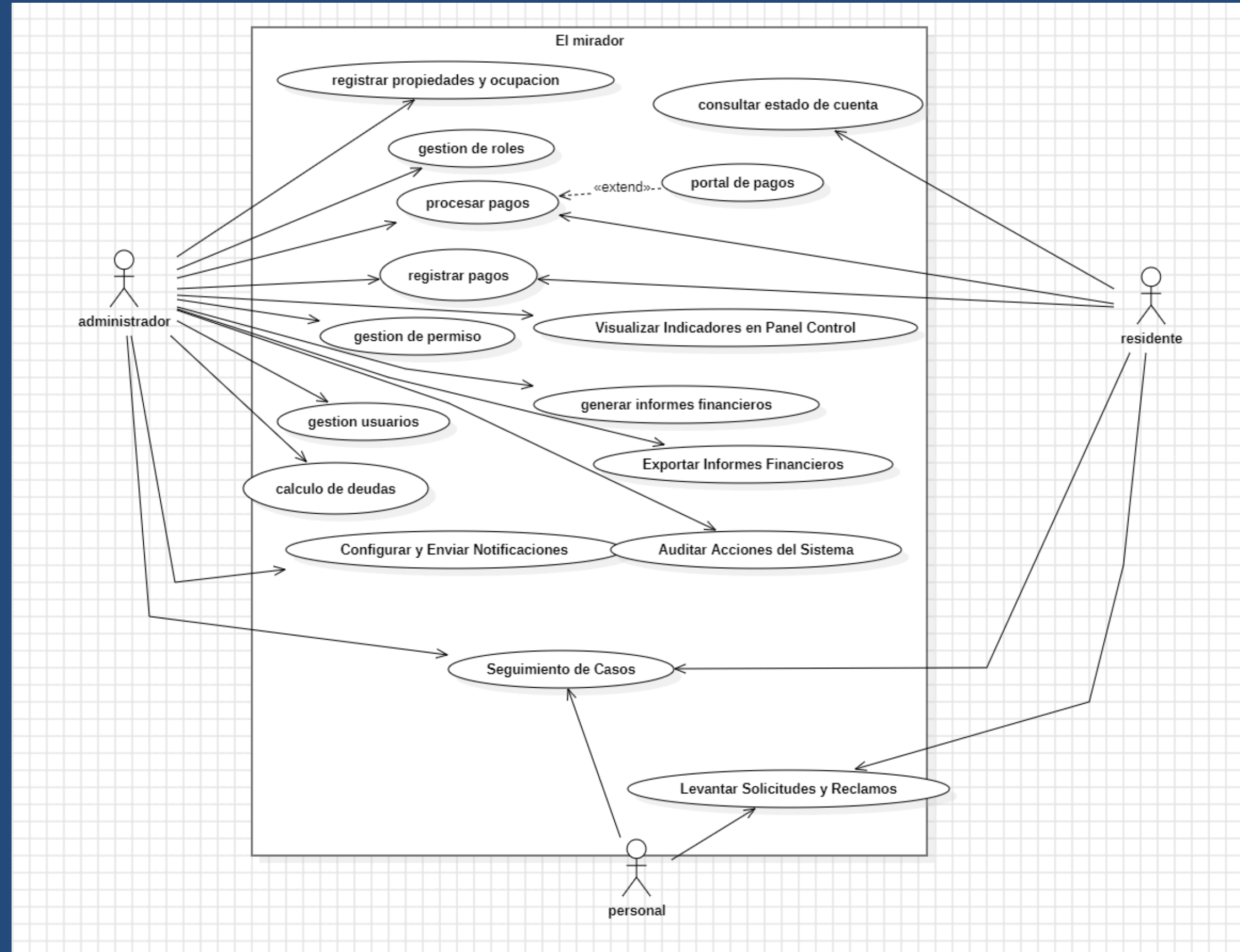
Estrategias de gestion

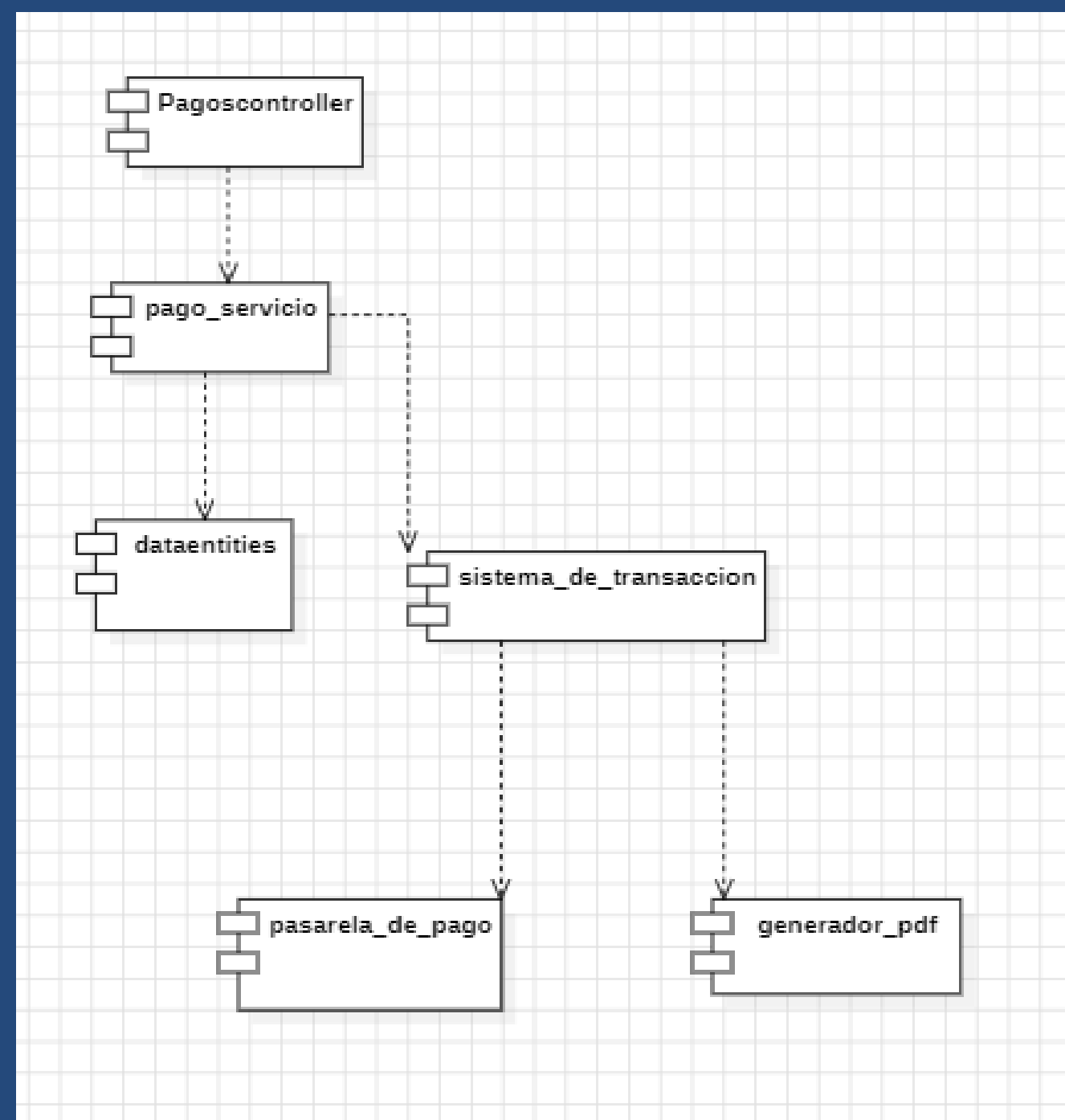
Co

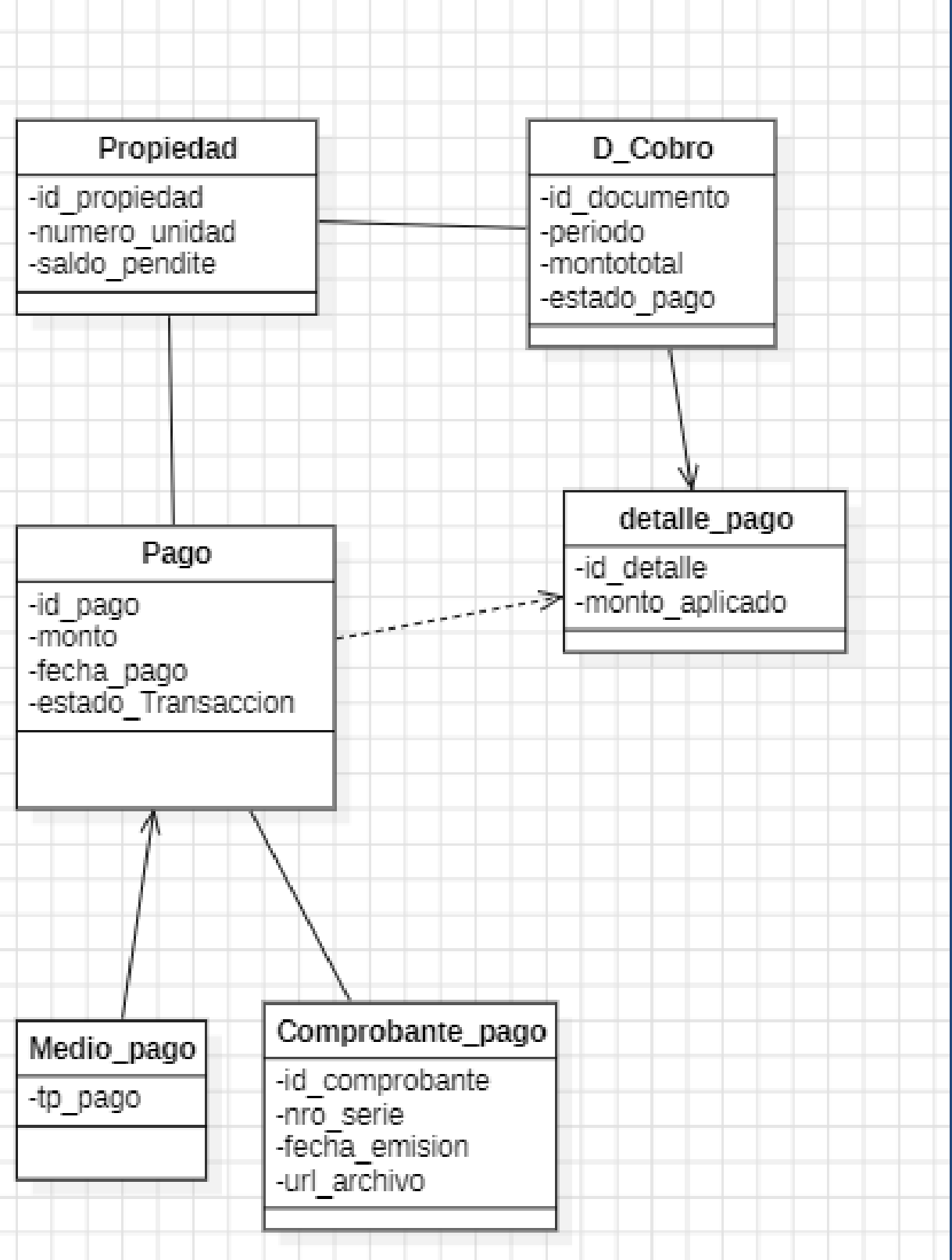
Carta Gantt del Proyecto

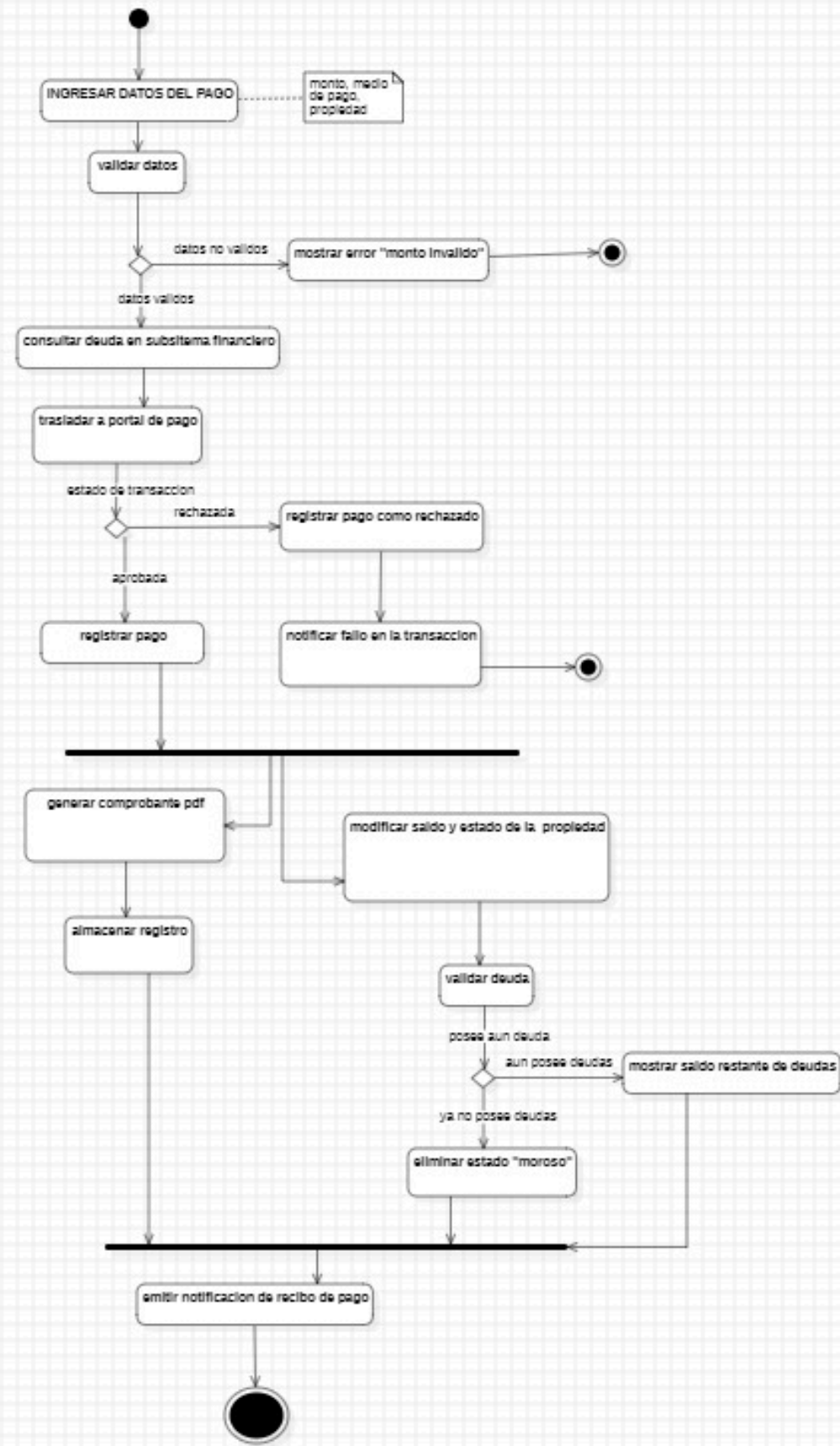
Sistema de Gestión de Pagos - Condominio El Mirador

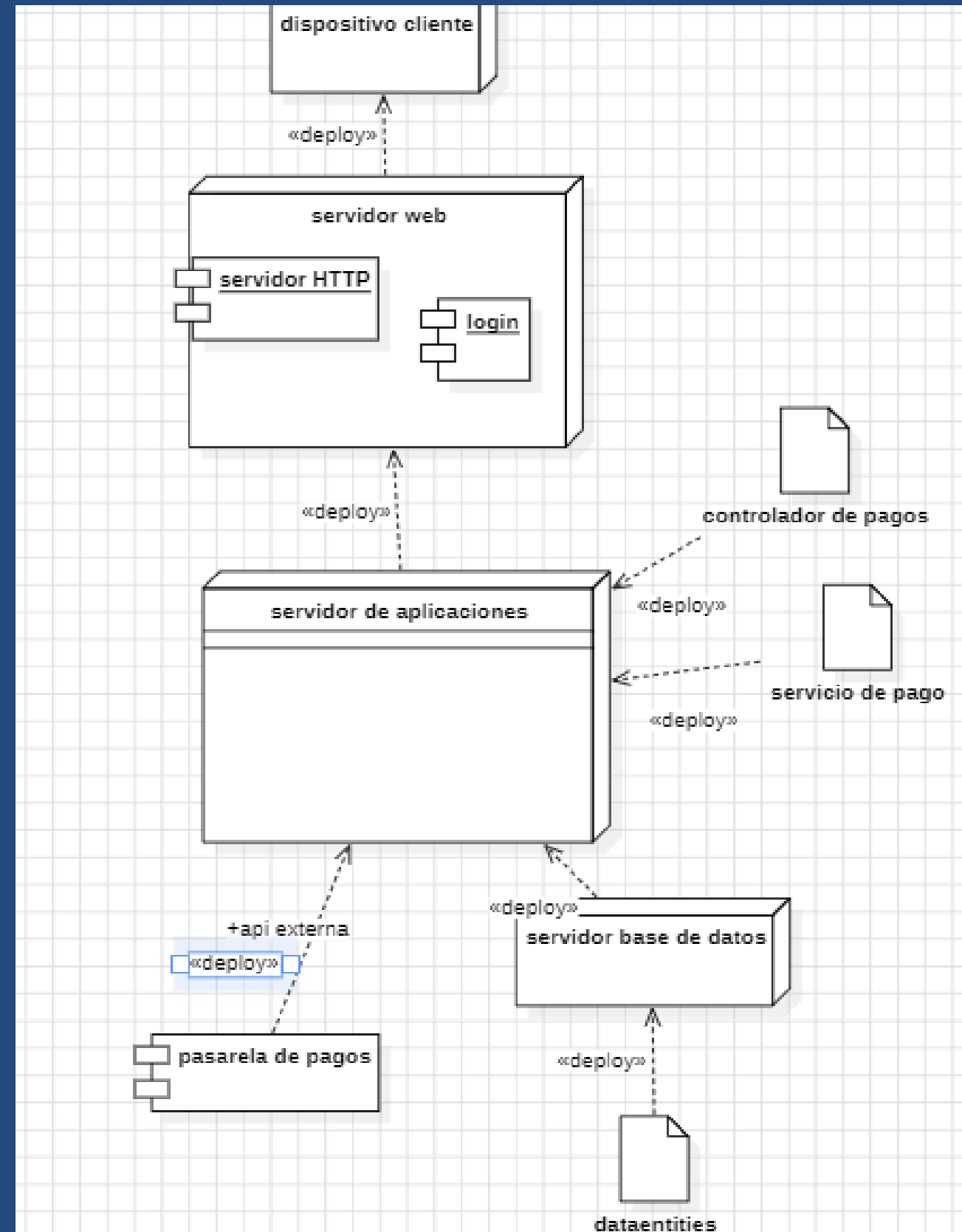
ID	Fase / Hito / Tarea	Duración Proyectada	Fecha de Inicio	Fecha de Término	Entregable / Hito Vinculado	Estado
1	FASE DE ANÁLISIS Y PLANIFICACIÓN	4 semanas	01/04/2026	28/04/2026	Hito: Lanzamiento Inicial	Finalizado
1.1	Levantamiento de Requerimientos y definición del contexto del problema (El Mirador).	1.5 semanas	01/04/2026	10/04/2026	Matriz de Requerimientos (RF/RNF).	Finalizado
1.2	Modelamiento del Alcance y Especificación de Casos de Uso del Subsistema de Pagos.	1.5 semanas	11/04/2026	21/04/2026	Casos de Uso (CU-001 a CU-005).	Finalizado
1.3	Definición de Arquitectura en Capas y Patrones de Diseño (MVC, Repositorio, Estrategia).	1 semana	22/04/2026	28/04/2026	Versión 1.0 (Prototipo Inicial Documental)	Finalizado
2	FASE DE DISEÑO Y PROTOTIPADO (SPRINT 1)					
	Diseño de Interfaces de Usuario en Figma (Panel Residente, Abonos y Consola Administrador).	2 semanas	29/04/2026	12/05/2026	Hito: Ajuste UX/UI por Feedback	Finalizado
2.1	Ejecución de Auditoría Técnica: Evaluación de Calidad Heurística de Nielsen.	1 semana	29/04/2026	04/05/2026	Mockups de alta fidelidad iniciales (v1.0.1).	Finalizado
2.2	Levantamiento de correcciones de bugs de interfaz y feedback de usuarios en Figma.	4 días	05/05/2026	08/05/2026	Matriz Excel de Evaluación Heurística.	Finalizado
2.3	FASE DE MODELADO TÉCNICO Y ARQUITECTURA (SPRINT 2)	4 días	09/05/2026	12/05/2026	Versión 1.0.2 (Gran actualización del sistema)	Finalizado
3	Construcción del Modelo Estático: Diagrama de Clases y lógica asociativa (Detalle_Pago).	1 semana	13/05/2026	18/05/2026	Hito: Cierre de Línea Base Arquitectura	Finalizado
3.1	Definición de la Vista de Implementación (Componentes y Paquetes cl.mirador.pagos).	2 días	13/05/2026	14/05/2026	Vista Lógica (Diagrama de Clases).	Finalizado
3.2	Modelado Dinámico y Físico (Diagramas de Actividad con Fork/Join concurrente y Despliegue).	2 días	15/05/2026	16/05/2026	Versión 1.1 (Última Revisión del Informe DAS)	Finalizado
3.3	FASE DE DESARROLLO E INTEGRACIÓN (SPRINTS 3 Y 4)	2 días	17/05/2026	18/05/2026	Línea Base Aprobada (Próxima revisión).	Finalizado
	Programación de la Capa de Datos (DataEntities) y Repositorios de persistencia en subred VPC.					
4	Desarrollo de la Lógica de Negocio (PagosService) y reglas de validación transaccional (Monto > 0).	4 semanas	19/05/2026	15/06/2026	Hito: Motor Transaccional Construido	Finalizado
4.1	Integración con APIs externas (Webpay/Flow/Khipu) bajo el patrón Estrategia.	1 semana	19/05/2026	26/05/2026	Base de Datos inicial aislada y segura.	Finalizado
4.2	FASE DE TESTING, CIERRE Y ENTREGA (SPRINT 5)	1.5 semanas	27/05/2026	05/06/2026	Motor lógico transaccional implementado.	Finalizado
4.3	Pruebas de Resistencia a la Concurrencia (Propiedades ACID) y renderizado asíncrono de PDFs.	1.5 semanas	06/06/2026	15/06/2026	Módulo de comunicación exterior (infraestructura_pago).	Finalizado
5	Despliegue final en la nube (AWS) y publicación del código modular en GitHub.	3 días	16/06/2026	18/06/2026	Hito: Prototipo Final y Sistema Funcional	Finalizado
5.1		2 días	16/06/2026	17/06/2026	Reporte de QA / Pruebas unitarias aprobadas.	Finalizado
5.2		1 día	18/06/2026	18/06/2026	Versión 1.2 (Prototipo final y sistema funcional modular).	Finalizado











Requisitos funcionales

El sistema debe permitir pagar deudas en línea, desglosar automáticamente un abono en el historial usando la clase `Detalle_pago`, y emitir comprobantes en PDF

Requisitos no funcionales: Seguridad (Infraestructura en subredes privadas VPC y cumpliendo PCI-DSS), Rendimiento (tiempos de respuesta menores a 2.5 segundos mediante procesos para correos y PDfs).

Metrica utilizada:
ISO25010 (Adecuacion funcional y fiabilidad).

Consistencia de datos:
Propiedades de ACID en la base de datos para evitar cobros duplicados o errores en los saldos ante las caidas de red.

Disponibilidad: SLA del 90% proyectado para el correcto funcionamiento del software.

Demostracion del prototipo de alta fidelidad

Para el desarrollo del prototipo de alta fidelidad se utilizó Figma, debido a su capacidad para crear interfaces interactivas colaborativas en tiempo real, permitiendo simular componentes reales, transiciones dinámicas y una experiencia de usuario (UX) idéntica a la aplicación final

Flujo 1: Entrada y Registro: El camino que hace el usuario para crearse una cuenta e iniciar sesión por primera vez.

Flujo 2: [El corazón de tu app - ej: Hacer una compra o Reservar una hora]: El proceso clave de la aplicación, desde que el usuario elige lo que busca hasta que el sistema le confirma que todo salió bien.

Flujo 3: Historial y Perfil: La sección donde el usuario puede revisar sus datos guardados y ver sus actividades anteriores.

Requisitos funcionales y no funcionales

- RF-01: Gestión de Autenticación y Acceso: Implementación de pantallas de Login, Recuperación de contraseña y Registro de usuarios, incluyendo validaciones visuales de campos obligatorios.
- RF-02: Control de Perfil de Usuario: Interfaz que permite al usuario visualizar, editar sus datos personales y configurar preferencias de la cuenta.
- RF-03: Módulo Principal [Adaptar según tu proyecto - ej: Gestión de Ventas / Reservas]: Flujo completo que permite al usuario [ej: buscar un producto, añadir al carrito, seleccionar método de pago y ver pantalla de éxito].
- RF-04: Visualización de Historial / Reportes: Pantalla con tablas o tarjetas interactivas que muestran los registros previos del usuario, permitiendo aplicar filtros o búsquedas dinámicas.

RNF-01: Usabilidad y Accesibilidad (UI/UX): Interfaz diseñada bajo la heurística de consistencia y estándares, utilizando una paleta de colores de alto contraste que cumple con la norma WCAG 2.1 (para legibilidad) y tipografías con escala jerárquica clara.

RNF-02: Diseño Responsivo (Adaptabilidad): Maquetación basada en sistemas de rejillas (Layout Grids) y componentes con Auto Layout en Figma, asegurando la adaptabilidad de la interfaz a resoluciones [ej: Mobile y Desktop / Web].

RNF-03: Intuitividad (Curva de aprendizaje): Flujos de navegación optimizados para completarse en un máximo de 3 clics desde la pantalla de inicio, reduciendo la carga cognitiva del usuario.

RNF-04: Identidad Visual y Consistencia: Aplicación estricta de un sistema de diseño (Design System) que define el uso unificado de botones, estados (hover, active, disabled), iconos y espaciados en todas las pantallas.

1. Enfoque del Plan de Pruebas (ISO 25000)

Explica brevemente bajo qué características de la norma evaluaron el prototipo de "El Mirador":

Adecuación Funcional: Se verificó que el sistema cumpla con las tareas clave (Control de acceso, Registro de encomiendas y Gestión de reservas).

Usabilidad: Evaluación de la facilidad de aprendizaje, estética de la interfaz en Figma y protección contra errores del usuario (campos obligatorios).

Seguridad: Simulación de restricciones para asegurar que un residente no pueda ver la información de otros departamentos ni acceder al panel del administrador.

CP-01: Registro Exitoso de Visitas

Pasos: Ingresar RUT válido, asignar Depto 402 y guardar.

Resultado: Exitoso (Passed) -> El sistema almacena la visita correctamente.

CP-02: Bloqueo de Reserva por Choque de Horario

Pasos: Intentar reservar el Quincho 1 en un bloque horario ya ocupado por otro departamento.

Resultado: Exitoso (Passed) -> El sistema despliega alerta bloqueando la acción.

CP-03: Notificación de Encomienda

Pasos: Registrar paquete recibido en conserjería.

Resultado: Exitoso (Passed) -> Se gatilla la alerta visual hacia la vista del residente.

Acciones de mejora y control de versiones

Pasarela de Pago Integrada: Pasar de la visualización estática del gasto común en el prototipo a una integración real con la API de Webpay o similares, permitiendo la conciliación bancaria automatizada para el administrador del edificio.

Sistema de Contingencia Offline: Diseñar un mecanismo de sincronización local para que la base de datos de los residentes y visitas autorizadas siga operando en la pantalla de conserjería si se cae el servicio de internet en el edificio.

Automatización de Notificaciones: Reemplazar las alertas manuales de encomiendas por un sistema automatizado de lecturas de códigos QR que dispare notificaciones push inmediatas al teléfono del residente apenas el paquete sea recibido.

Control de Versiones (Evidencia de GitHub)

Uso de Repositorio: Centralización del proyecto en GitHub (RQY1102--JoaquinFI-ET) para el trabajo colaborativo del equipo.

Trazabilidad: Registro formal de cambios mediante commits semánticos ordenados (desde la estructura inicial v1.0 hasta la organización final v1.3).

Hitos de Entrega: Creación de un Release v1.2 oficial en la plataforma, asegurando el respaldo y congelamiento del código del prototipo final entregado.